



AIN SHAMS UNIVERSITY FACULTY OF ENGINEERING

Verification project

CSE313s (UG2018)

Name	ID
Ahmed Tarek Hassanien	2200772
Mahmoud Ahmed Gamal	2200433
Sherif Ahmed Abdelfatah	2200176
Ahmed Emad Mohamed	2201374
Nour Ahmed Khalaf	2200176
Mariam Tarek Samir	2200167
Rana Gamal Reda	2200749

(TEAM 16)

SPI Master Verification Project

1. Project Overview

This project focuses on verifying an SPI Master design connected through an APB interface.

The verification environment was developed to validate:

- SPI protocol correctness
- APB register operations
- FIFO functionality
- Interrupt behavior
- Timing behavior
- Corner-case handling
- Randomized stability

The environment combines directed tests, randomized tests, assertions, functional coverage, BFM, and a reference model to ensure the correctness and robustness of the DUT.

2. Folder Structure & Verification Flow

Main Verification Components

Verification Flow

The verification process follows these steps:

1. Configure the DUT through the APB interface.
2. Generate SPI transactions.
3. Drive transactions using BFM.
4. Monitor SPI activity (MOSI, MISO, SCLK, CS).
5. Compare DUT outputs with the reference model.
6. Collect functional coverage.
7. Validate timing and protocol correctness using assertions.

3. Detailed Testcases Description

3.1 Sanity Test

File

sanity_test.sv

Objective

The sanity test is the first basic validation test used to confirm that the SPI Master DUT and the verification environment are functioning correctly.

It verifies that a standard SPI transfer can be completed successfully without any protocol or functional violations.

Verification Flow

1. Apply reset to initialize the DUT.
2. Configure SPI control registers through the APB interface.
3. Load transmit data into the TX register/FIFO.
4. Start SPI transfer.
5. Monitor SPI signals during transmission.
6. Wait for transfer completion interrupt.
7. Compare transmitted and received data.

Main Checks

- Correct APB register programming
- Proper SPI transfer initiation
- MOSI signal activity during transmission
- Correct SCLK generation
- BUSY signal assertion/deassertion timing
- Transfer completion interrupt generation
- Correct RX data reception

3.2 Register Access Test

File

reg_access_test.sv

Objective

The objective of reg_access_test.sv is to verify the correctness and robustness of the APB register interface of the SPI Master DUT.

This test validates proper register read/write behavior, reset functionality, access permissions (read-only/write-only), and DUT response to valid and invalid register operations.

The test serves as a foundational verification stage before moving to more advanced verification such as SPI modes, FIFO stress, interrupts, and randomized testing.

Verification Flow

1. Reset DUT.
2. Write values to each register.

3. Read back register contents.
4. Compare expected and actual values.
5. Verify reset default values.

Main Checks

- Register read/write consistency
- Control register configuration
- Status register updates
- Data register accessibility
- Register field correctness
- Reset values validation

Corner Cases Covered

- Multiple consecutive writes
- Read-after-write operations
- Partial register modifications

3.3 Randomized Register Access Test

File

randomized_reg_access_test.sv

Objective

The objective of randomized_reg_access_test.sv is to verify the SPI Master APB register interface under randomized register access scenarios.

The test applies seeded random APB transactions to improve functional coverage, validate register behavior, verify reset functionality, and ensure DUT stability under random register traffic and corner-case conditions.

Verification Flow

1. Generate randomized APB transactions.
2. Randomly select register addresses.
3. Randomly generate write data.
4. Perform back-to-back APB operations.
5. Monitor DUT response.

Main Checks

- Random register accesses
- Illegal/reserved address handling

- APB timing stability
- DUT robustness under random traffic
- No unexpected register corruption

Randomization Features

- Random addresses
- Random data patterns
- Random operation ordering
- Random delays between transactions

3.4 RAL Hardware Reset Test

File

ral_hw_reset_test.sv

Objective

Verifies that all DUT registers return to their specification-defined reset values after hardware reset assertion.

Verification Flow

1. Program registers with non-default values.
2. Assert reset signal.
3. Read all DUT registers.
4. Compare against expected reset values.

Main Checks

- Reset signal propagation
- Register initialization correctness
- Reset timing behavior
- DUT startup stability

4. SPI Protocol & Data Transfer Verification

4.1 Width Coverage Test

File

width_coverage_test.sv

Objective

Validates SPI transfers using all supported transfer widths.

Verification Flow

1. Configure transfer width.
2. Generate transfer data.
3. Start SPI transaction.
4. Monitor transaction completion.
5. Repeat for all supported widths.

Supported Widths

- 8-bit / 16-bit / 32-bit

Main Checks

- Correct transfer length
- Accurate bit shifting
- Proper BUSY timing
- Correct RX data reconstruction
- Transfer completion accuracy

4.2 Randomized Width Coverage Test

File

randomized_width_coverage_test.sv

Objective

Improves functional coverage by combining random payloads with randomized transfer widths.

Verification Flow

1. Randomly select transfer width.
2. Generate randomized data payload.
3. Execute multiple consecutive transfers.
4. Compare transmitted and received data.

Main Checks

- Random width switching
- Consecutive transfers
- Dynamic width transitions
- DUT stability across random traffic

Coverage Goals

- All width configurations exercised
- Multiple width transitions covered
- Random payload combinations covered

4.3 Mode Coverage Test

File

mode_coverage_test.sv

Objective

Validates SPI communication across all SPI operating modes.

SPI Modes Verified

1

Verification Flow

1. Configure CPOL and CPHA.
2. Start SPI transfer.
3. Monitor sampling and shifting edges.
4. Verify received data correctness.

Main Checks

- Correct clock polarity
- Correct clock phase
- Sampling edge accuracy
- Shift edge correctness
- Timing compliance

4.4 Loopback Test

File

loopback_test.sv

Objective

Validates internal data integrity using loopback operation.

Verification Flow

1. Enable loopback mode.
2. Send transmit data.
3. Internally route MOSI to RX path.
4. Capture received data.
5. Compare TX and RX values.

Main Checks

- TX-to-RX integrity
- Internal routing correctness

- Data corruption detection
- Timing consistency

Randomized Features

- Random SPI modes / widths / payload / delays

4.5 Delay Transfer Test

File

delay_transfer_test.sv

Objective

Verifies DUT stability when delays exist between SPI transfers.

Verification Flow

1. Perform SPI transfer.
2. Insert programmable delay.
3. Start next transfer.
4. Monitor DUT timing behavior.

Main Checks

- Transfer gap handling
- BUSY signal deassertion
- Restart timing
- Idle-state stability

Delay Variations

- Small /Large delays
- Randomized idle cycles
- Zero delay
- Single-transfer vs back-to-back transfers

5. Stress, Interrupt & Corner-Case Verification

5.1 FIFO Stress Test

File

fifo_stress_test.sv

Objective

Stress-tests FIFO operation under continuous heavy traffic conditions.

Verification Flow

1. Continuously push TX data.
2. Continuously read RX data.
3. Fill FIFO to maximum capacity.
4. Empty FIFO repeatedly.

Main Checks

- FIFO full condition
- FIFO empty condition
- Overflow protection
- Underflow protection
- Continuous throughput stability

Stress Conditions

- Back-to-back transfers
- High-frequency traffic
- Randomized transfer timing

5.2 Interrupt Test

File

interrupt_test.sv

Objective

Verifies the complete interrupt handling mechanism of the SPI Master DUT, including interrupt generation, masking, IRQ signaling, W1C clearing behavior, overflow events, and interrupt race-condition handling.

Interrupts Verified

- TX_EMPTY interrupt
- RX_FULL interrupt
- TX_OVF interrupt
- RX_OVF interrupt
- TRANSFER_DONE interrupt

Verification Flow

1. Generate different interrupt events.
2. Verify corresponding INT_STAT bits are asserted.
3. Check IRQ behavior with interrupts masked and unmasked.
4. Clear interrupts using W1C mechanism.

5. Verify interrupt deassertion after clearing.
6. Test simultaneous hardware-event and software-clear race conditions.

Main Checks

- Correct interrupt assertion behavior
- Proper IRQ masking/unmasking operation
- Accurate W1C interrupt clearing
- Correct interrupt status updates
- Overflow interrupt generation
- Stable interrupt behavior during race conditions
- No missing or false interrupt events

5.3 Clock Divider Corner Test

File

clk_div_corner_test.sv

Objective

Validates SPI clock generation across divider corner cases.

Verification Flow

1. Configure minimum divider.
2. Measure generated SCLK.
3. Configure maximum divider.
4. Repeat transfers across divider range.

Main Checks

- Clock frequency scaling
- SCLK stability
- Timing correctness
- Divider register functionality

Corner Cases

- Minimum divider values
- Maximum divider values
- Rapid divider switching

5.4 Error Injection Test

File

error_injection_test.sv

Objective

is to verify the SPI Master DUT behavior under invalid, illegal, and corner-case scenarios.

The test checks DUT robustness against FIFO overflow conditions, empty FIFO accesses, illegal configurations, and reserved APB address accesses while ensuring stable and safe DUT operation.

Verification Flow

1. Inject invalid APB accesses.
2. Configure illegal control combinations.
3. Attempt reserved register accesses.
4. Monitor DUT recovery behavior.

Main Checks

- Reserved address handling
- Illegal configuration protection
- Stable DUT recovery
- No internal corruption

Error Scenarios

- Invalid register offsets
- Unsupported configurations
- Illegal control settings

5.5 Randomized Sanity Test

File

randomized_sanity_test.sv

Objective

Performs long randomized end-to-end testing to maximize functional coverage and DUT robustness.

Verification Flow

1. Generate randomized configurations.
2. Generate randomized transfers.
3. Apply randomized timing variations.
4. Run extended simulation cycles.

Main Checks

- Random configuration stability
- Long-run DUT reliability
- Protocol correctness
- Data integrity
- Timing robustness

Randomization Features

- Random SPI modes
- Random transfer widths
- Random delays
- Random payloads
- Random transaction ordering

6. Coverage & Assertions

Coverage Files

- coverage.sv
- coverage2_regfile.sv
- coverage_spi_core.sv

objective

The objective of the functional coverage model is to ensure comprehensive verification of the SPI Master DUT by tracking all critical configuration combinations, protocol behaviors, FIFO operations, interrupt scenarios, reset conditions, clock divider configurations, loopback functionality, and APB register accesses.

The coverage environment measures whether all important functional scenarios and corner cases have been exercised during simulation to improve verification completeness and achieve the required functional coverage target.

Coverage Features

- Coverage for SPI modes, transfer widths, bit ordering, and loopback configurations.
- FIFO coverage including full, empty, and overflow conditions.
- Interrupt coverage for assertion and clearing behavior.
- APB register read/write and reset-value coverage.
- Clock divider and delay configuration coverage.
- Cross coverage used to verify important configuration combinations.
- Coverage integrated with directed and randomized tests to improve verification completeness.

Assertions (SVA)

File

spi_sva.sv

Assertion Checks

- Verify reset values after PRESETn assertion.
- Verify SPI disable behavior when CTRL.EN=0.
- Verify FIFO full/empty status behavior.
- Verify SS_n combinational behavior and assertion timing.
- Verify APB protocol timing and signal stability.
- Verify interrupt generation and W1C (Write-1-to-Clear) behavior.
- Verify TX/RX overflow flag assertion conditions.
- Verify reserved APB addresses return zero.
- Verify SCLK idle polarity matches configured SPI mode.
- Verify BUSY signal behavior during transfers.
- Verify SCLK divider timing and toggle behavior.
- Verify loopback routing functionality.
- Verify transfer configuration latching and stability during active transfers.
- Verify transfer gap (S_GAP) insertion behavior for delayed transfers.
- Verify SS_n remains asserted throughout active SPI transfers.

Simulation Method

The project simulation and verification flow were executed using a Makefile-based run flow.

The Makefile was used to compile the DUT and verification environment, run the required testcases, execute regression tests, and generate coverage reports automatically.

Different testcases were executed using Makefile commands, while randomized tests were run using different seeds to improve functional coverage and corner-case verification.

Makefile commands flow:

run from MSYS2

then invoke

make clean

make compile

make regress

make cov

7. Final Results & Conclusion

The verification environment successfully validated the SPI Master design using:

- Directed testing
- Randomized testing
- Assertions (SVA)
- Functional coverage
- Reference model checking

The project achieved strong confidence in:

- SPI protocol compliance
- Register functionality
- FIFO robustness
- Interrupt handling
- Timing correctness
- Corner-case behavior
- Overall DUT stability

The combination of BFMs, scoreboard, assertions, coverage, and randomized scenarios provided comprehensive verification for the SPI Master design.